Memory defenses

Mainack Mondal

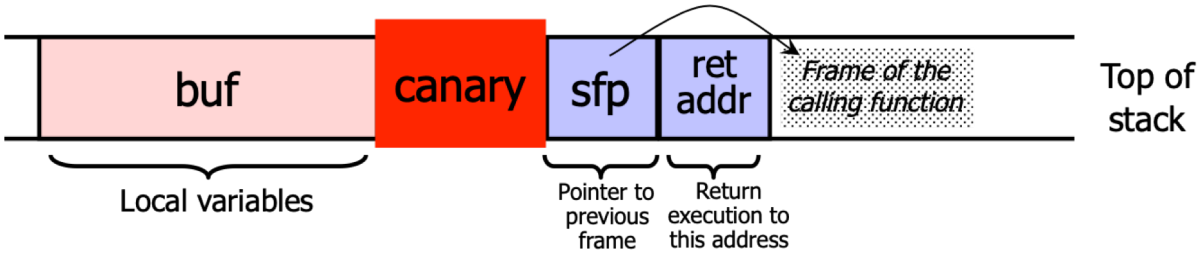
CS60112
Spring 2025

The road ahead

- Memory attacks
  - Buffer overflow
  - Ret2libc
  - Integer overflow
  - Format string vulnerabilities
  - Heap attacks
- Defenses
  - DEP
  - Stack canaries
  - ASLR
  - Reference monitor: Enforceable security policies

Stackguard

- Embed "canaries" (stack cookies) in stack frames and verify their integrity prior to function return
  - Any overflow of local variables will damage the canary



Three kinds of canary

- Terminator canary: “\0”, newline, linefeed, EOF
  - String functions like strcpy won’t copy beyond “\0”
  - Disadvantage: the canary is known
    ⇒the attacker can overwrite it with the known value
- Random canary: random string chosen on program start, stored in a global variable padded by unmapped pages
  - Attacker can’t guess what the value of canary will be and if it tries to exploit bugs to read off RAM, it will cause a segmentation fault
  - Disadvantage: the attacker can still learn the canary if it knows where to read it from

- Random XOR canary: random string chosen on program start, XOR scrambled using all or part of the control data (return pointer, etc.)
  - Same vulnerability as random canaries, the attacker needs to learn the canary AND the control data and the scrambling algorithm

Stack canaries

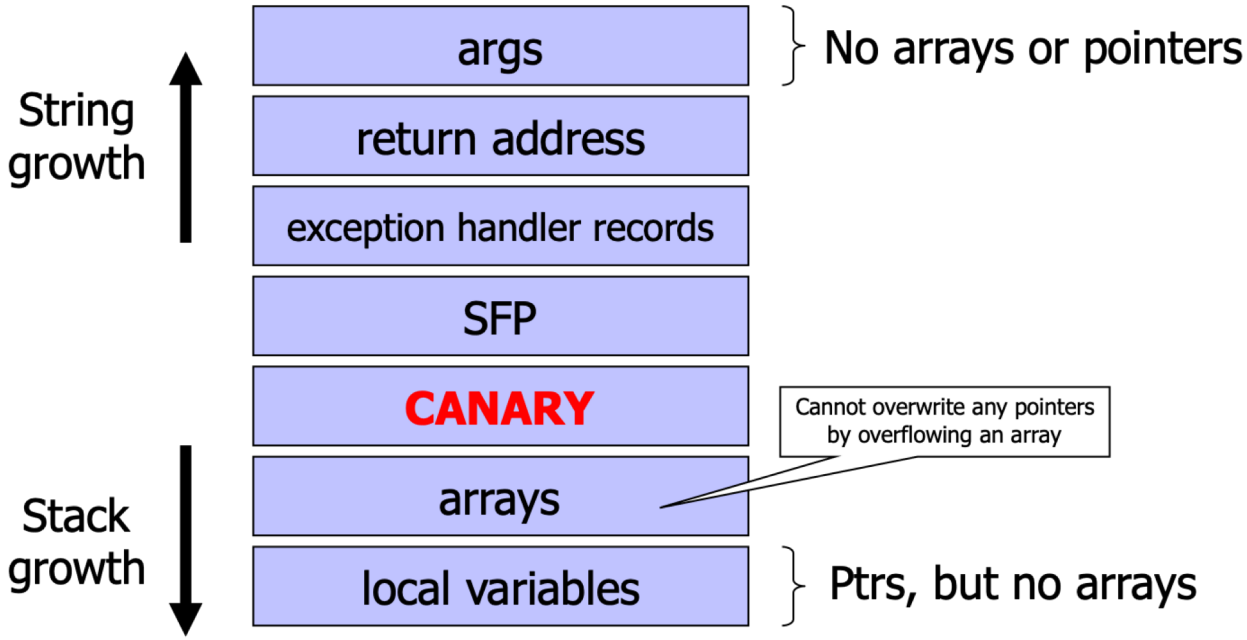
- Widely implemented
  - StackGuard (Crispin Cowan, GCC patch, 1997)
  - ProPolice (IBM)
    - first implemented as a GCC 3.x patch
    - included (reimplemented) in GCC 4.1 as “Stack-smashing Protection” (SSP)
    - -fstack-protector GCC flag
    - standard in OpenBSD, FreeBSD, and variants of Linux (e.g. Ubuntu)
  - /GS flag for MS Visual Studio compiler (since 2003)
- Very small overhead (a few percent)
  - Only needed on functions with local arrays
  - Even so, with Windows /GS not always applied (heuristics)
    - Not a good idea: ANI attack on Vista (2007)
    - struct vs. array

Stackguard implementation

- StackGuard requires code recompilation
- Checking canary integrity prior to every function return causes a performance penalty
  - For example, 8% for Apache Web server
- StackGuard can be defeated
  - A single memory copy where the attacker controls both the source and the destination is sufficient

ProPolice/SSP [IBM, used in gcc 3.4.1; also MS compilers]

- Rerrange stack layout (requires compiler mod)

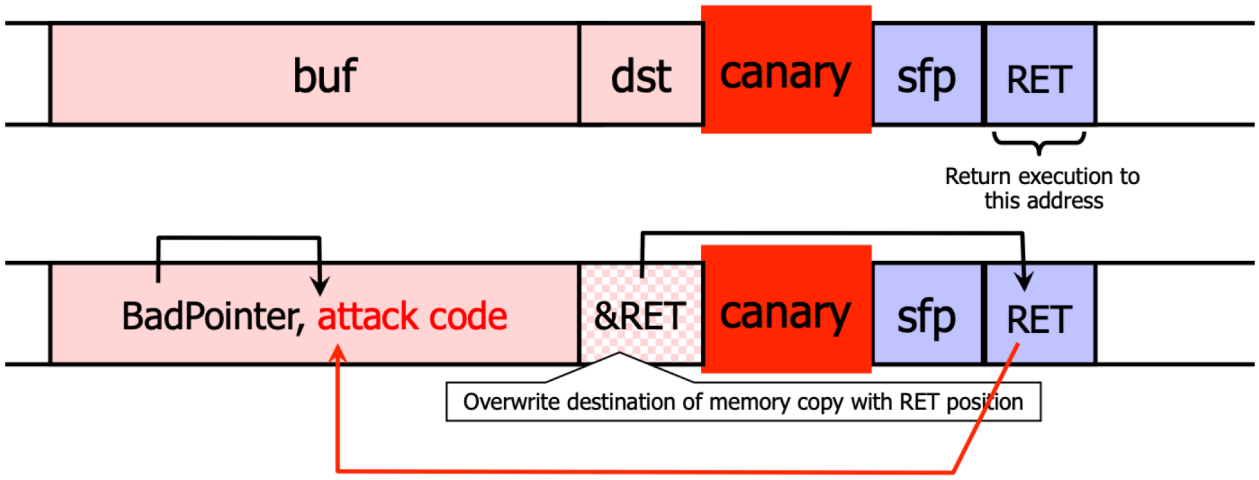


Stack canaries: limitations

- Do not prevent heap-based buffer overflows
- Only protect against contiguous buffer overflows
  - Won't detect if exploit writes to arbitrary address directly
- No protection if attack happens before function returns
  - Canary won't detect if exploit overwrites
    - argument function pointer that gets called before function returns
    - exception handler that gets invoked before function returns
- Canary alone offers no protection for local pointers
  - They are allocated before the canary
  - Bad in particular for function pointers, but not only
- Still, good as a first barrier of defense

Defeating Stackguard

- Suppose program contains *dst=buf[0] where attacker controls both dst and buf
  - Example: dst is a local pointer variable



What can still be overwritten?

- Other string buffers in the vulnerable function
- Any data stored on the stack
  - Exception handling records
  - Pointers to virtual method tables
    - C++: call to a member function passes as an argument "this" pointer to an object on the stack
    - Stack overflow can overwrite this object's vtable pointer and make it point into an attacker-controlled area
    - When a virtual function is called, control is transferred to attack code
    - Do canaries help in this case?
      (Hint: when is the integrity of the canary checked?)

Litchfield's Attack

- Microsoft Windows 2003 server implements several defenses against stack overflow
  - Random canary (with /GS option in the .NET compiler)
  - When canary is damaged, exception handler is called
  - Address of exception handler stored on stack above RET
- Attack: smash the canary and overwrite the pointer to the exception handler with the address of the attack code
  - Attack code must be on heap and outside the module or else Windows won't execute the fake "handler"
  - Similar exploit used by CodeRed worm

Code Red Worm (2001)

- A malicious URL exploits buffer overflow in a rarely used URL decoding routine in MS-IIS …
- … the stack-guard routine notices the stack has been smashed, raises an exception, calls handler
- … pointer to exception handler located on the stack, has been overwritten to point to CALL EBX instruction inside the stack-guard routine
- … EBX is pointing into the overwritten buffer
- … the buffer contains the code that finds the worm's main body on the heap and executes it

Safe Exception Handling

- Exception handler record must be on the stack of the current thread
- Must point to a valid handler
  - Microsoft's /SafeSEH linker option: header of the binary lists all valid handlers
- Exception handler records must form a linked list, terminating in FinalExceptionHandler
  - Windows Server 2008: SEH chain validation
  - Address of FinalExceptionHandler is randomized

Otherwise it is easy to fake the exception handler and place it in a "valid" list

SEHOP

- SEHOP: Structured Exception Handling Overwrite Protection (since Win Vista SP1)
- Observation: SEH attacks typically corrupt the “next” entry in SEH list
- SEHOP adds a dummy record at the top of the SEH list
- When exception occurs, dispatcher walks up list and verifies dummy record is there; if not, terminates process
- Can also be bypassed:
  - http://dl.packetstormsecurity.net/papers/general/SEH-all-at-once-attack.pdf

The road ahead

- Memory attacks
  - Buffer overflow
  - Ret2libc
  - Integer overflow
  - Format string vulnerabilities
  - Heap attacks
- Defenses
  - DEP
  - Stack canaries
  - ASLR
  - Reference monitor: Enforceable security policies

Problem: lack of diversity

- Classic memory exploits need to know the (virtual) address to hijack control
  - Address of attack code in the buffer
  - Address of a standard kernel library routine
- Same address is used on many machines
  - Slammer infected 75,000 MS-SQL servers in 10 minutes using identical code on every machine (buffer overflow in Microsoft's SQL server)
- Idea: introduce artificial diversity
  - Make stack addresses, addresses of library routines, etc. unpredictable and different from machine to machine

ASLR

- Address Space Layout Randomization
- Randomly choose base address of stack, heap, code segment, location of Global Offset Table
  - Randomization can be done at compile- or link-time, or by rewriting existing binaries
- Randomly pad stack frames and malloc'ed areas
- Other randomization methods: randomize system call ids or even instruction set
- Implemented in today's OSs
  - Mac OS
    - 10.5 (October 2007) for system libraries
    - 10.7 (July 2011) for all applications
    - 10.8 (July 2012) for kernel, loadable kernel modules, etc. (at boot time)
  - Android (since 4.0), iOS (since 4.3), Linux (since 2.6.12), Windows (since Vista), …

Base-address randomisation

- Only the base address is randomized
  - Layouts of stack and library table remain the same
  - Relative distances between memory objects are not changed by base address randomization
- To attack, it's enough to guess the base shift
- A 16-bit value can be guessed by brute force
  - Try 2^{15} (on average) overflows with different values for addr of known library function – how long does it take?
    - In "On the effectiveness of address-space randomization" (CCS 2004), 2^{16} seconds to compromise Apache
  - If address is wrong, target will simply crash

ASLR in Windows

- Vista and Server 2008
- Stack randomization
  - Find Nth hole of suitable size (N is a 5-bit random value), then random word-aligned offset (9 bits of randomness)
- Heap randomization: 5 bits
  - Linear search for base + random 64K-aligned offset
- EXE randomization: 8 bits
  - Preferred base + random 64K-aligned offset
- DLL randomization: 8 bits
  - Random offset in DLL area; random loading order

- For further information:
  - Nice reading on DEP and ASLR http://security.stackexchange.com/questions/18556/how-do-aslr-and-dep-work

Example: ASLR in Vista

- Booting Vista twice loads libraries in different locations:

ntlanman.dll	0x6D7F0000	Microsoft® Lan Manager
ntmarta.dll	0x75370000	Windows NT MARTA provider
ntshrui.dll	0x6F2C0000	Shell extensions for sharing
ole32.dll	0x76160000	Microsoft OLE for Windows

ntlanman.dll	0x6DA90000	Microsoft® Lan Manager
ntmarta.dll	0x75660000	Windows NT MARTA provider
ntshrui.dll	0x6D9D0000	Shell extensions for sharing
ole32.dll	0x763C0000	Microsoft OLE for Windows

- ASLR is only applied to images for which the dynamic-relocation flag is set

Bypassing Windows ASLR

- Implementation uses randomness improperly, thus distribution of heap bases is biased
  - Ollie Whitehouse, Black Hat 2007
  - Makes guessing a valid heap address easier
- When attacking browsers, the attacker
  - may be able to insert arbitrary objects into the victim's heap
    - Executable JavaScript code, plugins, Flash, Java applets, ActiveX and .NET controls, DLLs
  - may exploit components (e.g., DLLs) not enabling ASLR
    - see vulneabilities in McAffee and Symantec http://www.scriptjunkie.us/2011/06/bypassing-dep-aslr-in-browser-exploits-with-mcafee-symantec/
- Heap spraying
  - Stuff heap with multiple copies of attack code

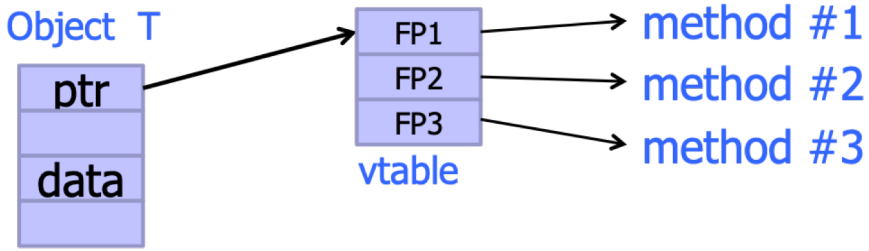
Defeating ASLR: Exploit Heap

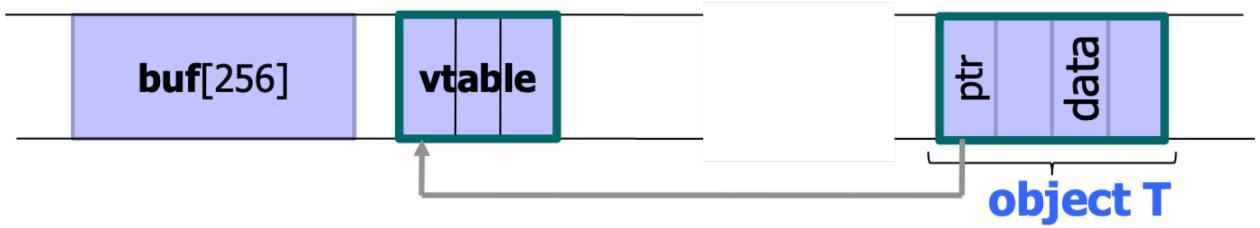
Heap overflow

- Overflowing buffers on heap can change pointers that point to important data
  - Illegitimate privilege elevation: if program with overflow has sysadm/root rights, attacker can use it to write into a normally inaccessible file
    - Example: replace a filename pointer with a pointer into a memory location containing the name of a system file (for example, instead of temporary file, write into AUTOEXEC.BAT)
- Sometimes can transfer execution to attack code
  - Example: December 2008 attack on XML parser in Internet Explorer 7 - see http://isc.sans.org/diary.html?storyid=5458

Function pointers on the heap

- Compiler-generated function pointers (e.g., virtual method table in C++ or JavaScript code)

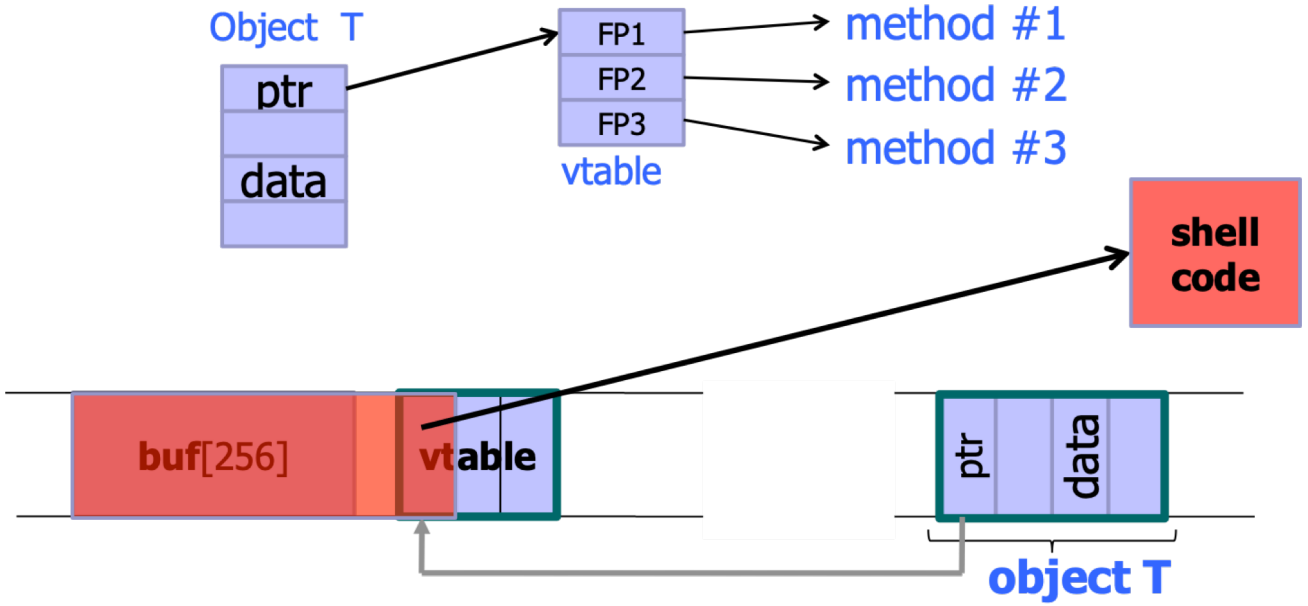




- Suppose vtable is on the heap next to a string object

Heap-based control hijacking

- Compiler-generated function pointers (e.g., virtual method table in C++ or JavaScript code)

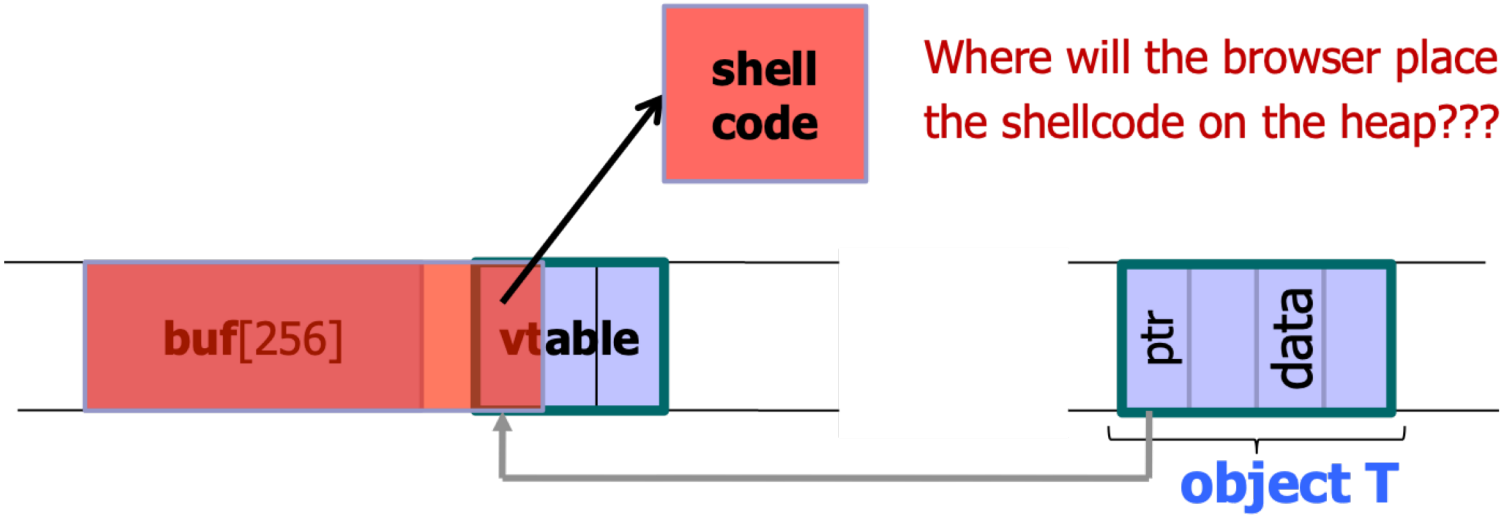


- Suppose vtable is on the heap next to a string object

Problem?

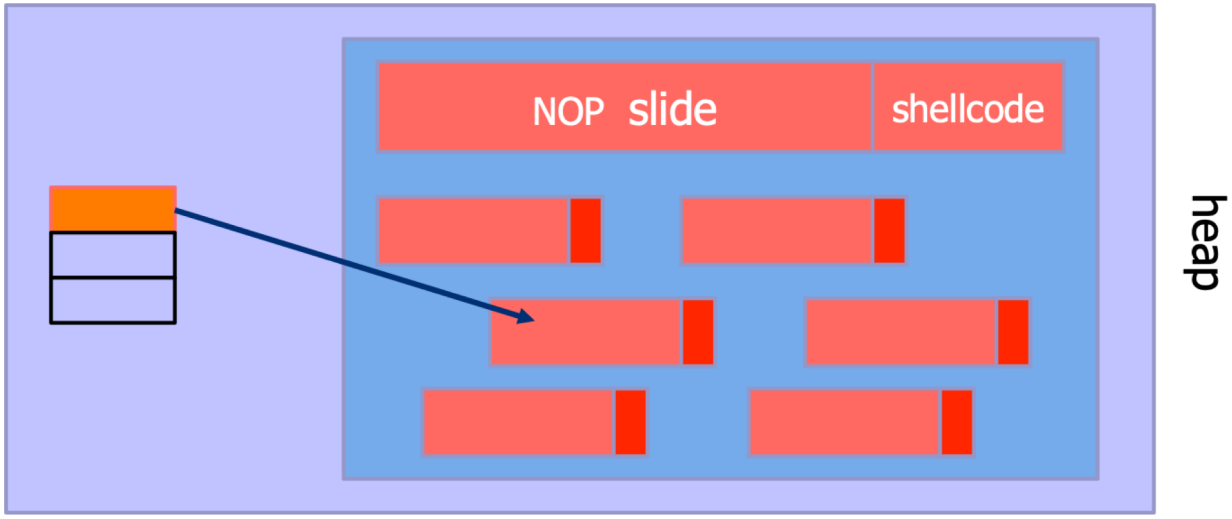
```
<SCRIPT language="text/javascript">
      shellcode = unescape("%u4343%u4343%...");
      overflow-string = unescape("%u2332%u4276%...");

      cause-overflow( overflow-string );      // overflow  buf[ ]
</SCRIPT?
```



Heap spraying

- Force JavaScript JiT (“just-in-time” compiler) to fill heap with executable shellcode, then point SFP or vtable ptr anywhere in the spray area



JavaScript heap spraying

- Pointing a function pointer anywhere in the heap will cause shellcode to execute

```
var  nop = unescape("%u9090%u9090")

while (nop.length < 0x100000)  nop += nop

var shellcode = unescape("%u4343%u4343%...");

var x = new Array ()
for (i=0;  i<1000;  i++) {
     x[i] = nop + shellcode;
}
```

- The code is generated locally, since sending it over the Internet would be impractical
- Also, tradeoff between program usability (the user might close the window if she has to wait too long) and likelihood to hit the sprayed area

So far we saw use of address randomization

How about code randomization?

In-place code randomization

Smashing the Gadgets: Hindering Return-Oriented Programming Using In-Place Code Randomization

Vasilis Pappas
Columbia University
vpappas@cs.columbia.edu

Michalis Polychronakis
Columbia University
mikepo@cs.columbia.edu

Angelos D. Keromytis
Columbia University

IEEE S&P'12

- Instruction reordering

```
MOV EAX, &p1
MOV EBX, &p2
```

→

```
MOV EBX, &p2
MOV EAX, &p1
```

- Instruction substitution

```
MOV EBX, $0
```

→

```
XOR EBX, EBX
```

- Register re-allocation

```
MOV EAX, &p
CALL *EAX
```

→

```
MOV EBX, &p
CALL *EBX
```

Instruction location randomization

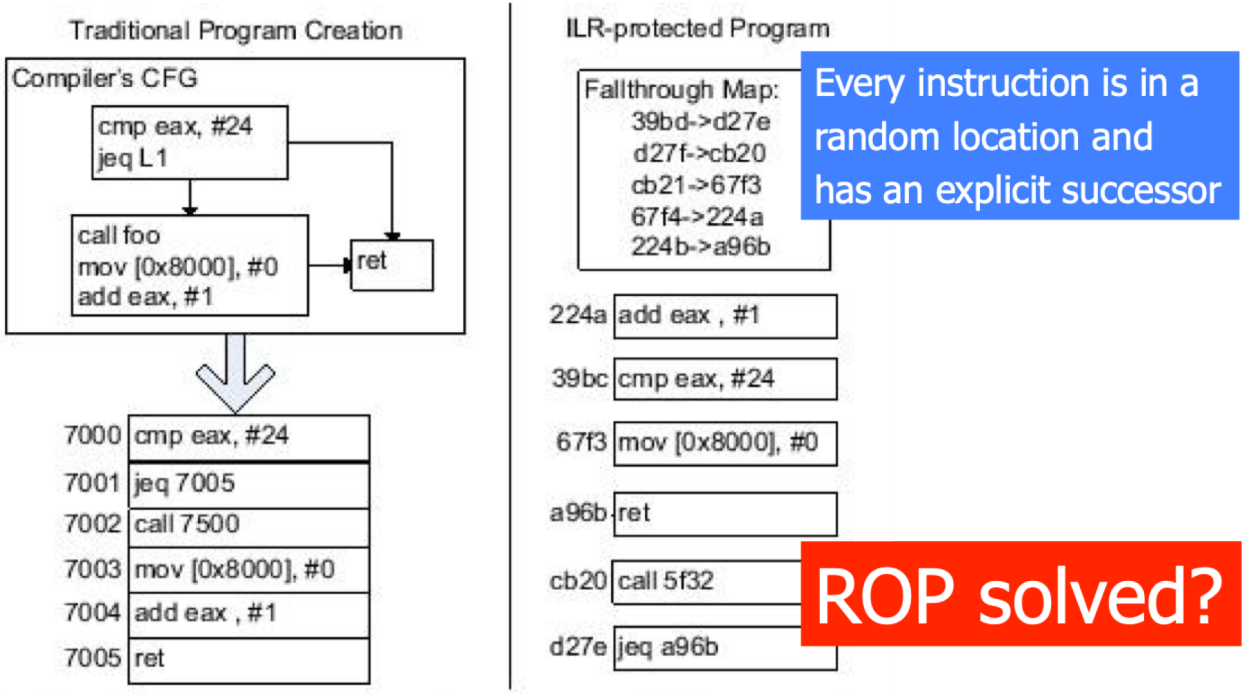
Smashing the Gadgets: Hindering Return-Oriented Programming Using In-Place Code Randomization

Vasilis Pappas
Columbia University
vpappas@cs.columbia.edu

Michalis Polychronakis
Columbia University
mikepo@cs.columbia.edu

Angelos D. Keromytis
Columbia University

IEEE S&P'12



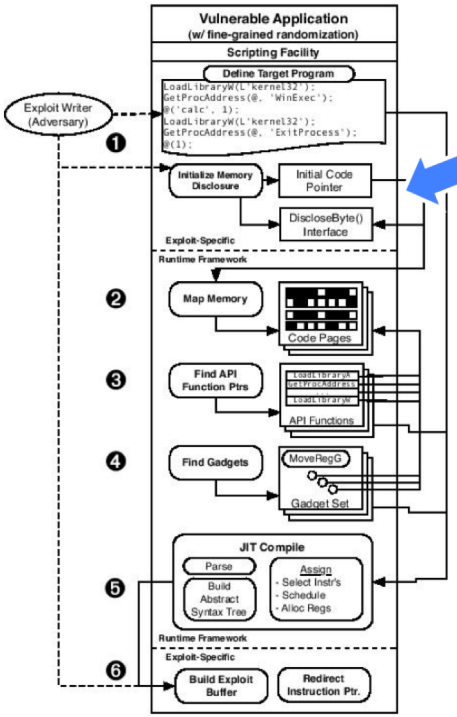
Just-in time code reuse

Just-In-Time Code Reuse:
On the Effectiveness of Fine-Grained Address Space Layout Randomization

Kevin Z. Snow, Fabian Monrose
Department of Computer Science
University of North Carolina at Chapel Hill, USA
Email: {kzsnow,fabian}@cs.unc.edu

Lucas Davi, Alex
Christopher Liebchen,
CASED/Technische Universität Da
Email: {lucas.davi,alexandra.dmitrien
christopher.liebchen,ahmad.sadeghi}@trust.cased.de

IEEE S&P'13



Find one code pointer
(using any disclosure vulnerability)

The entire page must be code…
Analyze the instructions to find
jumps and calls to other code pages…

Map out a big portion of
the application's code pages

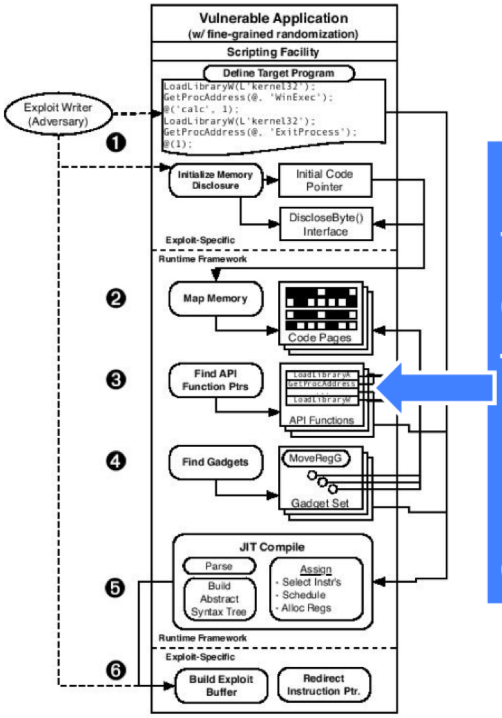
Just-in time code reuse

Just-In-Time Code Reuse:
On the Effectiveness of Fine-Grained Address Space Layout Randomization

Kevin Z. Snow, Fabian Monrose
Department of Computer Science
University of North Carolina at Chapel Hill, USA
Email: {kzsnow,fabian}@cs.unc.edu

Lucas Davi, Alex…
Christopher Liebchen, …
CASED/Technische Universität Da…
Email: {lucas.davi,alexandra.dmitrien…
christopher.liebchen,ahmad.sadeghi}@trust.cased.de

IEEE S&P'13



Use typical opcode sequences to find calls to LoadLibrary() and GetProcAddr()…
These can be used to invoke any library function by supplying the right arguments - don't need to discover the function's address!

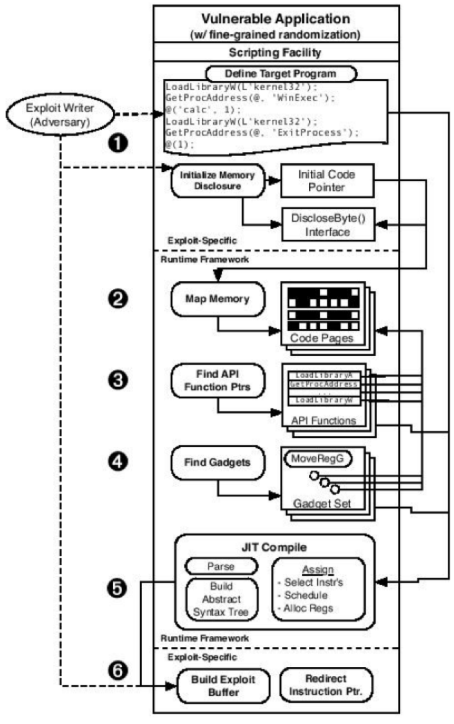
Just-in time code reuse

Just-In-Time Code Reuse:
On the Effectiveness of Fine-Grained Address Space Layout Randomization

Kevin Z. Snow, Fabian Monrose
Department of Computer Science
University of North Carolina at Chapel Hill, USA
Email: {kzsnow,fabian}@cs.unc.edu

Lucas Davi, Alex…
Christopher Liebchen, …
CASED/Technische Universität Da…
Email: {lucas.davi,alexandra.dmitrie…
christopher.liebchen,ahmad.sadeghi}@trust.cased.de

IEEE S&P'13



Collect gadgets in runtime by analyzing the discovered code pages (dynamic version of Shacham's "Galileo" algorithm)

Compile on the fly into shellcode

Overall

- ASLR is a first line defense
- Can be defeated by heaps (if not randomized)
- Additional reading
  - Blind ROP attack in presence of ASLR
  - Hacking Blind, Bittau et al.
  - https://www.scs.stanford.edu/brop/bittau-brop.pdf

The road ahead

- Memory attacks
  - Buffer overflow
  - Ret2libc
  - Integer overflow
  - Format string vulnerabilities
  - Heap attacks
- Defenses
  - DEP
  - Stack canaries
  - ASLR
  - Reference monitor: Enforceable security policies